Два приложения: GitHub Pages + Gradio Space

Создаём сайт, запускаем Gradio на своём компьютере, затем публикуем Python-приложение в Hugging Face Spaces и сравниваем два способа запуска.

РЕЗУЛЬТАТ ЗАНЯТИЯ

1. Публичная визитка USERNAME.github.io с двумя отдельными коммитами: HTML/CSS, затем JavaScript.
2. Локальный Gradio: Python-приложение открывается на localhost и перестаёт работать после остановки процесса.
3. Публичный Gradio Space: студент самостоятельно создаёт файлы приложения, отправляет их через Git и проверяет опубликованную страницу.
4. Студент объясняет, где выполняется Python в локальной и опубликованной версиях.

Что подготовить

- Учётные записи GitHub и Hugging Face; заранее проверьте вход и подтверждение адреса электронной почты.
- Windows 10/11, браузер, Git, редактор файлов, Python 3.12 и Gradio. Установка описана в технической памятке.
- Свой код: напишите его самостоятельно или создайте с помощью любого доступного AI-помощника. Доступ к конкретному сервису не требуется.

ПРАВИЛО КУРСА

Исходники создаются и изменяются локально. Передача кода на GitHub и Hugging Face — только командами Git в терминале. Веб-интерфейс HF используйте для создания пустого Space и выбора оборудования; файлы приложения через браузер не загружайте.

Планируйте около 2–3 часов, особенно при первой установке Python и пакетов. Незавершённые шаги можно закончить дома.

Маршрут и требования

Сначала выполните статическую публикацию и локальный Gradio. Публикация в HF требует отдельного публичного Space и отправки кода через Git.

Этап	Что делает студент	Как проверяем
А1 · сайт	HTML/CSS-визитка, первый commit и push	Рабочий адрес USERNAME.github.io
А2 · JS	Добавляет JavaScript отдельным коммитом	Интерактивный элемент и история Git
Б1 · локально	Создаёт app.py и запускает Gradio	Работающий localhost:7860 и остановка
Б2 · HF Space	Создаёт ZeroGPU Space и отправляет файлы через Git	Публичный Space с рабочими кнопками

Условия бесплатного HF Space

ПРОВЕРЬТЕ АККАУНТ ЗАРАНЕЕ

Бесплатный ZeroGPU для личного Gradio Space доступен аккаунтам HF в хорошем состоянии: адрес электронной почты подтверждён, аккаунт создан более 30 дней назад. На бесплатном аккаунте можно разместить до двух ZeroGPU Spaces. Если ZeroGPU недоступен, не выбирайте платное оборудование: сообщите преподавателю до занятия.

Минимальные требования к приложению

- Лабиринт автоматически создаётся в Python; размер задаётся элементами интерфейса Gradio.
- Кнопка показывает кратчайший путь от входа к выходу; пустые и граничные значения обрабатываются без аварийной остановки.
- В приложении нет внешних API, секретных ключей и тяжёлых моделей. Алгоритмы генерации и поиска используют CPU.

БЕЗ ОБЯЗАТЕЛЬНОГО CODEX

Любой студент может написать код сам или использовать любой доступный помощник по программированию. Задание содержит требования и шаги проверки, но не готовые файлы приложения.

Первая публикация визитки

01 Создайте HTML/CSS с помощью ИИ

Попросите два файла — **index.html** и **styles.css** — без JS и backend. Откройте index.html двойным кликом и проверьте страницу.

ПРИМЕР ЗАПРОСА К ИИ

«Создай аккуратный адаптивный сайт-визитку: “Обо мне”, две карточки проектов, контакты. Только HTML и CSS, без JavaScript, React, CDN и сервера. Покажи код index.html и styles.css и инструкции локальной проверки».

02 Создайте репозиторий

На GitHub создайте **Public**-репозиторий с точным именем **USERNAME.github.io** (USERNAME — ваш логин). Не добавляйте README, .gitignore и лицензию при создании: начните с пустого репозитория.

03 Выполните команды из папки с файлами

Коммиты создаются на компьютере. Задайте автора **только для текущего репозитория**.

```
git init -b main
git config user.name "Student Name"
git config user.email "email@example.com"
git status
git add index.html styles.css
git commit -m "Create static homepage"
git remote add origin https://github.com/USERNAME/USERNAME.github.io.git
```

04 Отправьте коммит

На общем ПК используйте временный токен и отключите helper сохранения учётных данных. В HTTPS-запросе укажите логин GitHub и fine-grained PAT вместо пароля.

```
git -c credential.helper= push -u origin main
```

05 Опубликуйте Pages

Repository → Settings → Pages → Deploy from a branch → main → /(root) → Save. Откройте **https://USERNAME.github.io/**. Публикация может занять несколько минут.

Добавляем JavaScript новым коммитом

Теперь ваш статический сайт должен получить одну функцию, работающую **в браузере**. Действие пользователя меняет DOM, и новый запрос к Python-серверу для этого не нужен.

Идея	Что происходит
Фильтр карточек	«Все / Учёба / Хобби» показывает нужные карточки.
Переключатель темы	Меняет тему и текст кнопки.
Список целей	Добавляет запись и обновляет счётчик на странице.
Живой предпросмотр	Изменение поля ввода обновляет приветствие.

ВТОРОЙ ЗАПРОС К ИИ

«У меня уже есть index.html и styles.css. Не переписывай их с нуля. Добавь отдельный script.js с фильтром карточек, объясни, как подключить его в HTML. Учти клавиатуру и отсутствие подходящих карточек».

Проверьте локально

- Кнопка или поле работают; меняется содержание страницы, не только оформление по hover.
- После обновления страницы интерфейс по-прежнему работает; проверьте также пустые или ошибочные значения.

Второй коммит и push

```
git status
git diff
git add index.html styles.css script.js
git commit -m "Add browser-side interactivity"
git -c credential.helper= push origin main
git log --oneline -3
```

ПРОВЕРКА

На опубликованной странице появился новый элемент. В истории GitHub есть как минимум два ваших коммита. Не создавайте второй коммит через веб-редактор GitHub.

GRADIO / B1

Запускаем обычный Gradio локально

Для этого этапа нужны **Python и пакет gradio**. Их установка, в том числе без прав администратора через uv, описана в отдельной технической памятке. Она же поможет, если localhost не открывается.

01 **Подготовьте окружение**
Выполните раздел «Python / Gradio» технической памятки: создайте .venv и установите gradio. Не загружайте .venv в GitHub или Hugging Face.

02 **Создайте app.py**
Попросите ИИ написать небольшой инструмент с двумя параметрами и понятным выводом. Пример на следующей странице.

03 **Запустите локальный сервер**
Откройте терминал в папке с app.py и выполните:

```
.\.venv\Scripts\python.exe app.py
```

04 **Проверьте страницу**
Откройте **http://127.0.0.1:7860** (или адрес, который вывел Gradio). Попробуйте два разных ввода и пустое значение.

05 **Остановите сервер**
Нажмите **Ctrl+C** в терминале и обновите страницу. Она перестанет отвечать: Python выполнялся отдельным серверным процессом.

ПОДРОБНЫЕ КОМАНДЫ

Установка Python 3.12 через uv, проверка pip/Gradio, действия при блокировке приложений и диагностика запуска вынесены в отдельный PDF.

Создайте приложение-лабиринт

Вместо готового исходника в задании описаны требования. Напишите приложение сами или попросите любой доступный инструмент помочь с кодом.

Часть	Что должно происходить
Генерация	Python автоматически создаёт случайный проходимый лабиринт алгоритмом DFS с возвратом.
Интерфейс	Gradio показывает карту, принимает ширину, высоту и необязательный seed, содержит кнопки создания и решения.
Решение	Поиск в ширину (BFS) находит кратчайший путь от входа до выхода; путь отображается поверх карты.
Проверки	Размеры ограничены разумным диапазоном; повторная генерация меняет лабиринт; решения существует; ошибки ввода не завершают сервер.

Порядок самостоятельной работы

- Сначала реализуйте генерацию и поиск пути как обычные Python-функции в maze.py.
- Затем создайте Gradio Blocks в app.py и соедините кнопки с функциями.
- Отрисуйте результат, например, как SVG; проверьте локально несколько размеров и seed.
- Эту же структуру файлов и функцию позже опубликуйте в собственном Gradio Space.

БЕЗ ГОТОВЫХ ФАЙЛОВ

Курс не раздаёт app.py, maze.py и другие исходники решения. Их создаёт студент: вручную или при помощи любого доступного AI-помощника. Полный запрос для генерации и критерии проверки приведены на странице 9.

На локальном этапе приложение выполняется на компьютере. Для Hugging Face потребуется README.md с настройкой Space и отдельный push через Git.

Что происходит после нажатия кнопки?

Используйте одно и то же приложение в локальной и опубликованной версиях. Обычный Gradio в обоих случаях запускает Python на сервере; меняется место, где этот сервер работает.

Характеристика	Локальный Gradio	Gradio Space на ZeroGPU
Где Python	В отдельном процессе на компьютере студента	В контейнере Hugging Face
Путь запроса	Браузер → localhost → Gradio → Python-функция	Браузер → HF Space → Gradio → Python-функция
Остановка	После Ctrl+C страница больше не отвечает	Работает по опубликованному адресу, пока Space запущен
Оборудование	CPU компьютера	ZeroGPU предоставляет GPU по запросу; алгоритм лабиринта работает на CPU

Особенность ZeroGPU

НУЖЕН ТЕХНИЧЕСКИЙ ОБРАБОТЧИК

Самому лабиринту GPU не требуется. Однако ZeroGPU при старте проверяет наличие обработчика Gradio с декоратором @spaces.GPU. Попросите генератор добавить скрытый callback с duration=1, зарегистрировать его на невидимой кнопке и отключить для него API. Этот callback не должен вызываться кнопками лабиринта. Если его не вызывать, вычисления генерации и решения остаются на CPU.

Ответьте на вопросы

- Где исполняется функция генерации в локальной и опубликованной версии?
- Почему локальная страница перестаёт отвечать после Ctrl+C?
- Почему ZeroGPU нужен для бесплатного Gradio-хостинга, хотя лабиринт использует CPU?
- Какие действия отправляют код на HF и что делать с токеном после работы?

Создайте Gradio Space

Сначала создайте пустой репозиторий Space в браузере, затем склонируйте его и добавьте файлы локально.

01 **Проверьте право на ZeroGPU**

Убедитесь, что адрес HF подтверждён, аккаунту больше 30 дней, а лимит бесплатных Spaces не исчерпан. Не выбирайте платный тариф.

02 **Создайте Space**

Hugging Face → New Space. Укажите своё имя и название Space, выберите Public, SDK Gradio и оборудование ZeroGPU. Если выбор оборудования находится в настройках, откройте Settings → Hardware → ZeroGPU.

03 **Склонируйте репозиторий**

Замените HF_USER и SPACE_NAME на свои значения. Команды выполняются в PowerShell или Git Bash.

```
git clone https://huggingface.co/spaces/HF_USER/SPACE_NAME
cd SPACE_NAME
git config user.name "Student Name"
git config user.email "email@example.com"
```

Создайте README.md

В шапке Space укажите SDK, версию Gradio и главный файл. Например:

```
---
title: My Maze
sdk: gradio
sdk_version: 6.31.0
python_version: "3.12"
app_file: app.py
---
```

ПРОВЕРЬТЕ ОБОРУДОВАНИЕ

После создания Space откройте Settings → Hardware и убедитесь, что указано ZeroGPU (идентификатор zero-a10g). Одна YAML-шапка в README не заменяет настройку оборудования. Не выбирайте CPU Upgrade или платный GPU.

В клоне Space далее создайте app.py, maze.py, requirements.txt и README.md локально. Веб-редактор HF для загрузки исходников не используйте.

Создайте файлы и запустите локально

Файлы приложения студенты создают сами: вручную или с помощью любого доступного AI-помощника. Готового решения в курсе нет.

ЗАПРОС ПОМОЩНИКУ ПО ПРОГРАММИРОВАНИЮ

Напиши учебное приложение на Python с Gradio 6.31 для генерации и решения лабиринтов.
Создай в текущей папке файлы app.py, maze.py, requirements.txt и README.md. Покажи каждый файл полностью и объясни, как сохранить его в редакторе.

ТРЕБОВАНИЯ К ПРИЛОЖЕНИЮ
- Генерируй случайный совершенный лабиринт алгоритмом DFS с возвратом.
- Находи кратчайший путь от входа до выхода поиском в ширину (BFS).
- Показывай лабиринт как SVG в Gradio Blocks. Добавь размеры, seed, кнопки «Создать» и «Решение».
- Раздели алгоритмы и интерфейс между maze.py и app.py; не используй модель или внешний API.
- requirements.txt закрепи на gradio==6.31.0. В README добавь HF YAML-шапку с sdk: gradio, sdk_version: 6.31.0, python_version: "3.12", app_file: app.py.
- ZeroGPU требует зарегистрированный обработчик @spaces.GPU. Добавь отдельный скрытый callback с duration=1 и зарегистрируй его на невидимой кнопке с api_name=False. Никогда не вызывай его из генерации или решения. Добавь локальный fallback-декоратор, если пакет spaces не установлен.
- Проверь размеры и пустые значения. Не включай секреты, токены, .venv или готовые примеры файлов.

После вывода файлов объясни, что именно создаёт лабиринт, как работает BFS и какие команды нужны для локального запуска и Git push в мой Space.

Проверьте файлы перед запуском

- В папке Space есть app.py, maze.py, requirements.txt и README.md; нет .venv и токенов.
- В app.py обработчики кнопок генерации и решения не используют @spaces.GPU.
- В README указаны sdk: gradio и app_file: app.py; Gradio закреплён на версии 6.31.0.

Установите зависимости и запустите проект

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe app.py
```

Откройте адрес, который напечатал Gradio, обычно http://127.0.0.1:7860. Создайте несколько лабиринтов разных размеров, покажите решение, затем остановите сервер через Ctrl+C.

Отправьте код и проверьте Space

Перед push убедитесь, что вы находитесь в клоне Space, а не в папке другого проекта.

Зафиксируйте только нужные файлы

```
git status
git add README.md app.py maze.py requirements.txt
git diff --cached
git commit -m "Publish maze Gradio app"
git -c credential.helper= push -u origin main
```

Авторизация HF

- Создайте Hugging Face User Access Token с правом записи в нужный Space; задайте срок действия.
- При запросе Git укажите HF-логин, а в поле пароля — токен. Не вставляйте токен в URL, команды, исходники или README.
- На общем компьютере отзовите временный токен после push и проверьте Windows Credential Manager.

Проверьте публикацию

01 **Дождитесь сборки**

Откройте https://huggingface.co/spaces/HF_USER/SPACE_NAME и дождитесь статуса Running on ZERO.

02 **Проверьте приложение**

Нажмите создание лабиринта, измените размер и нажмите «Решение». Убедитесь, что маршрут виден и сообщение содержит число шагов.

03 **Проверьте историю**

В разделе Files проверьте список файлов, а в Git history — свой commit. Исходники редактировались и отправлялись через Git.

ЕСЛИ ЗАПУСК УПАЛ

Проверьте Settings → Hardware: выбран ли ZeroGPU; build/container logs: нет ли ошибки No @spaces.GPU function detected during startup; список файлов: доступны ли app.py и requirements.txt. Если аккаунт не имеет доступа к ZeroGPU, остановитесь и обратитесь к преподавателю — не выбирайте платное оборудование.

БЕЗОПАСНОСТЬ / ОБЯЗАТЕЛЬНО

Отзовите токены перед уходом

Если вы работаете на общем университетском компьютере, необходимо не только закончить push, но и **отозвать временные токены доступа**. Это обязательный шаг практики.

01 **Не сохраняйте пароль или токен**
Для GitHub и Hugging Face используйте токены с минимальными правами, не вставляйте их в команды или URL. Выполняйте **git -c credential.helper= push** — команда отключает credential helper для этого запуска Git.

02 **Отзовите GitHub-токен**
GitHub → Settings → Developer settings → Personal access tokens. Найдите токен, созданный для занятия, и выберите **Revoke/Delete** (название зависит от типа токена).

03 **Отзовите Hugging Face-токен**
Hugging Face → Settings → Access Tokens. Найдите временный токен для отправки в Space и отзовите/удалите его.

04 **Завершите сеанс**
Выйдите из GitHub и Hugging Face, закройте окна браузера, удалите рабочие файлы на общем ПК. Если авторизация ранее была сохранена, очистите её в Windows Credential Manager. Не используйте постоянные SSH-ключи на чужом компьютере.

ВАЖНО

Отзыв токенов обязателен, даже если вы удалили папку проекта и закрыли браузер.
Закрытие приватного окна не делает сохранённый где-либо токен недействительным.

Подробный чек-лист безопасности и действия для чистой Windows есть в отдельной технической памятке.

Что сдавать и как оценивается

Отправьте преподавателю одно сообщение со ссылками, историей Git и кратким сравнением локального и удалённого запуска.

```
GitHub repo: https://github.com/USERNAME/USERNAME.github.io
Pages:       https://USERNAME.github.io/
HF Space:    https://huggingface.co/spaces/HF_USER/SPACE_NAME
2 commits:   SHA_INITIAL / SHA_JS
HF commit:   SHA_HF_SPACE
```

На очной проверке запустите app.py локально, покажите работу генерации и решения, остановите сервер Ctrl+C. Откройте HF Space и повторите действия в опубликованной версии.

Критерий	Проверка	Баллы
Статический сайт	Первая версия Pages и локальный commit + push	20
JavaScript	Реакция на действие и отдельный второй commit	20
Локальный Gradio	Запуск app.py, корректная генерация, поиск пути и остановка	25
HF Gradio Space	Публичный ZeroGPU Space, приложение работает, push через Git	20
Архитектура и безопасность	Объяснение исполнения Python, токены не сохранены, нет секретов	15

Ключевые вопросы

- Где выполняется Python локально и где — в HF Space?
- Почему Space может спать и почему локальное приложение зависит от запущенного процесса?
- Зачем ZeroGPU-handler, если алгоритм лабиринта не использует GPU?
- Как вы отправили код и обезопасили HF-токен?

Официальные материалы: Hugging Face Spaces — https://huggingface.co/docs/hub/spaces-overview · ZeroGPU — https://huggingface.co/docs/hub/en/spaces-zerogpu · Gradio Quickstart — https://www.gradio.app/guides/quickstart